



INSTITUTO TECNOLÓGICO DE BUENOS AIRES

PRIMER PARCIAL

Cloud Provider Analytics (ETL + Streaming + Serving en Cassandra)

Perez de Gracia, Mateo (63401)

Quian Blanco, Francisco (63006)

Stanfield, Theo (63403)

Mosquera, Diego

Big Data - 72.80

Primer cuatrimestre 2026

Tabla de contenidos

1	Introducción	2
2	Diagrama de arquitectura de alto nivel	3
2.1	Patrón Arquitectónico: Lambda	4
2.2	Zonas del Data Lake	4
3	Mapeo de Requisitos a Componentes	5
3.1	Requisitos Analíticos (Consultas de Negocio)	5
3.2	Requisitos de Negocio y Técnicos → Diseño	5
3.3	Las 5Vs del Big Data y cómo las cubre la solución	5
4	Flujos de Datos (Data Pipelines)	7
4.1	Pipeline Batch (Maestros y Facturación)	7
4.2	Pipeline Streaming (Eventos de Uso)	7
5	Supuestos y Riesgos Iniciales	8
5.1	Supuestos	8
5.2	Riesgos y Mitigaciones	8
6	Estimación de esfuerzo y recursos	9
7	Conclusión	10

1. Introducción

Este documento presenta el diseño arquitectónico preliminar para el proyecto *Cloud Provider Analytics*. El objetivo principal de esta entrega es permitir una **validación temprana** y evaluar la viabilidad de la solución técnica, priorizando un enfoque conciso y accionable que evite la sobre-ingeniería en esta fase inicial.

La propuesta plantea la construcción de un pipeline de datos *end-to-end* desarrollado con **PySpark** en Google Colab, utilizando **Parquet** como formato de almacenamiento intermedio estructurado en zonas lógicas (Data Lake), y **AstraDB (Cassandra)** como capa de servicio (Serving Layer).

Para satisfacer las distintas exigencias de latencia del negocio, el sistema implementa un patrón **Lambda**. Esta arquitectura permite procesar eficientemente tanto las cargas de trabajo por lotes (procesamiento *batch* para maestros y facturación) como el flujo continuo de eventos operativos (procesamiento *streaming near real-time* para eventos de uso), dejando los datos listos para ser consumidos por los equipos de FinOps, Soporte y Producto.

2. Diagrama de arquitectura de alto nivel

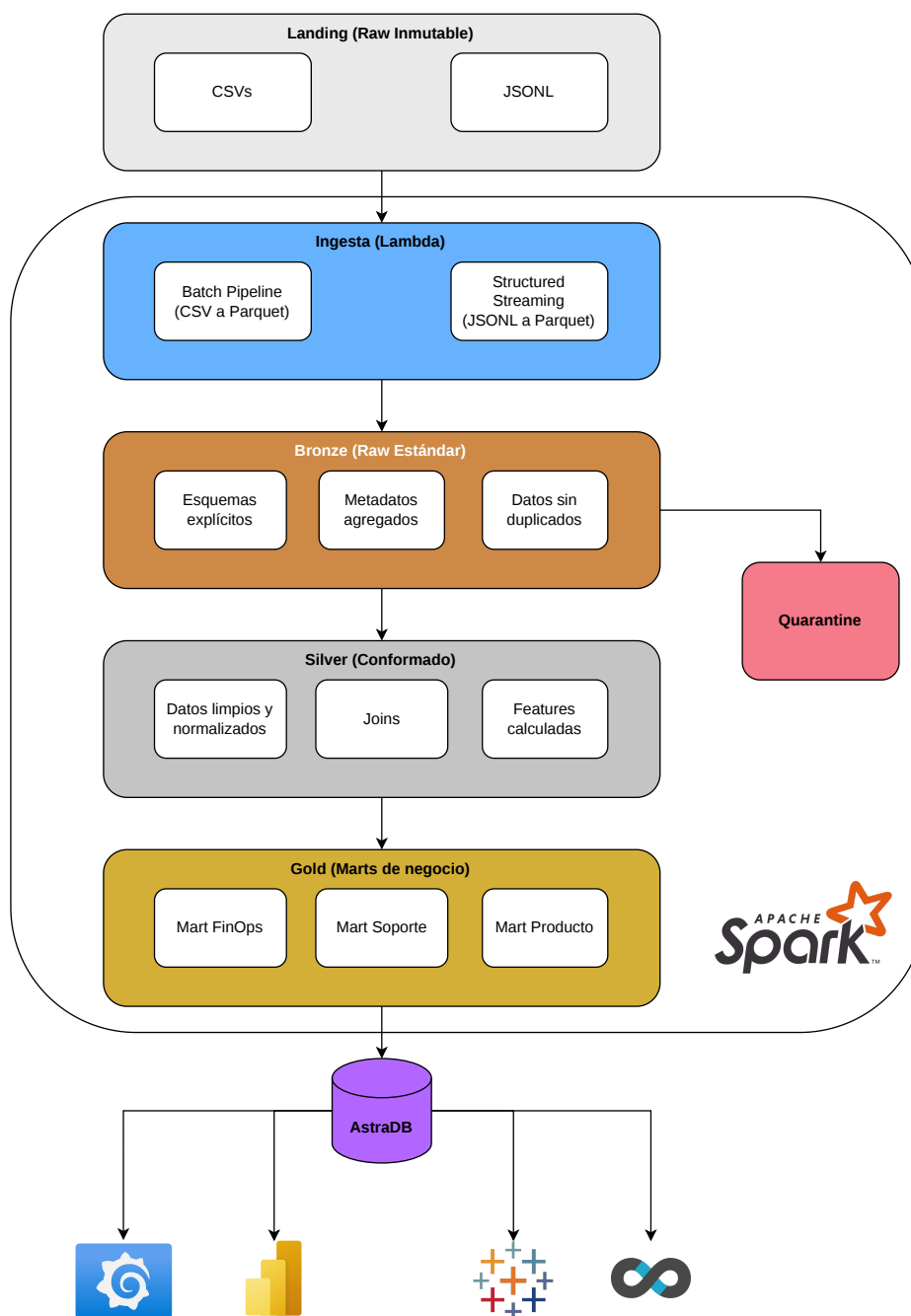


Figura 1: Diagrama de arquitectura de alto nivel

2.1. Patrón Arquitectónico: Lambda

Se elige el patrón **Lambda** para resolver la doble necesidad del negocio: procesamiento histórico complejo y baja latencia operativa.

- **Batch Layer:** Procesa datos pesados e históricos, como la facturación mensual (`billing_monthly.csv`), los maestros estáticos y las encuestas.
- **Speed Layer (Streaming):** Utiliza PySpark *Structured Streaming* para ingestar eventos de uso (`usage_events_stream` en `.jsonl`) en micro-lotes (*near real-time*). Gestiona la llegada de datos tardíos con *watermarks* y evita duplicados filtrando por `event_id`.

2.2. Zonas del Data Lake

El almacenamiento se estructura bajo el modelo Bronze-Silver-Gold, utilizando **Parquet particionado** gestionado por Spark.

- **Landing (Raw Inmutable):** Punto de entrada. Almacena los archivos originales (`.csv` estáticos y `.jsonl` de eventos) sin ninguna modificación.
- **Bronze (Raw Estándar):** Conserva la granularidad original. Aplica tipificación explícita de datos, deduplicación inicial por `event_id` y añade metadatos de auditoría (`ingest_ts`, `source_file`).
 - *Quarantine (Cuarentena):* Aísla en un directorio Parquet separado los registros con errores técnicos o que no cumplen reglas estrictas de calidad (ej. `event_id` nulo o tipos de datos malformados) detectados en la transición de Bronze a Silver.
- **Silver (Conformado):** Capa de limpieza y normalización. Realiza cruces (*joins*) con tablas de dimensiones, gestiona nulos y maneja la evolución del esquema (compatibilizando las versiones 1 y 2 ante la aparición de `carbon_kg` y `genai_tokens`).
 - *Detección de Anomalías:* A diferencia de los datos corruptos que van a cuarentena, las excepciones lógicas de negocio (ej. un pico inusual de consumo o costo) se calculan en esta capa utilizando percentiles como métodos estadísticos. Estos registros se conservan y se les agrega un *flag* de anomalía para que sigan su curso.
- **Gold (Marts de Negocio):** Datos agregados bajo un enfoque *query-first*. Contiene los marts de **FinOps** (que recibe los *flags* en su `cost_anomaly_mart`), **Soporte** y **Producto/Usage** optimizados y listos para ser exportados a la Serving Layer (AstraDB/Cassandra).

3. Mapeo de Requisitos a Componentes

3.1. Requisitos Analíticos (Consultas de Negocio)

Para asegurar que el diseño de la Serving Layer sea verdaderamente accionable y cumpla con el modelo *query-first* requerido en AstraDB, la arquitectura garantiza los *Marts* necesarios en la capa Gold para responder eficientemente las siguientes consultas críticas de los dashboards:

1. Costos y solicitudes (*requests*) diarios por organización y por servicio en un rango de fechas determinado.
2. Top-N de servicios según su costo acumulado en los últimos 14 días para una organización específica.
3. Evolución diaria de tickets críticos y la tasa de incumplimiento de SLA (*SLA breach*) durante los últimos 30 días.
4. Ingresos (*revenue*) mensuales totales, contemplando créditos e impuestos aplicados, normalizados a USD.
5. Cantidad de tokens de GenAI consumidos por día y su respectivo costo estimado.

3.2. Requisitos de Negocio y Técnicos → Diseño

Requisito Clave	Componente / Decisión de Diseño
Near real-time para métricas operativas (uso/consumos)	PySpark Structured Streaming leyendo el directorio <code>usage_events_stream/*.jsonl</code> , manejando datos tardíos (<i>late data</i>) con <code>withWatermark</code> y deduplicación por <code>event_id</code> .
Batch diario/mensual para maestros y facturación	PySpark Batch leyendo archivos <code>.csv</code> inmutables desde Landing para procesarlos hacia la capa Bronze.
Almacenamiento intermedio y particionado	Almacenamiento intermedio en formato Parquet particionado (por ejemplo, por <code>date=</code> y/o <code>service=</code>) gestionado íntegramente por Spark.
Datos crudos con inconsistencias y nulos	Implementación de reglas de validación estrictas entre Bronze y Silver; los registros que fallan se desvían a un directorio Quarantine en Parquet aparte.
Evolución de esquema a mitad del histórico	Compatibilización de versiones (<code>schema_version v1</code> y <code>v2</code>) en la capa Silver para integrar armónicamente los nuevos campos <code>carbon_kg</code> y <code>genai_tokens</code> .
Detección de anomalías en costos	Transformaciones en Silver utilizando métodos estadísticos (Z-score, MAD o percentiles) para generar un <i>flag</i> de anomalía en costos atípicos.
Idempotencia (reprocesar sin duplicar)	Uso de <i>checkpointing</i> , claves naturales y <i>upserts</i> en los flujos de escritura para garantizar la consistencia ante reprocesos.
Serving y consultas para Dashboards (BI)	AstraDB (Cassandra) con tablas modeladas bajo un enfoque <i>query-first</i> , cargadas usando el conector de Spark para Cassandra o <code>foreachBatch</code> + driver Python.

3.3. Las 5Vs del Big Data y cómo las cubre la solución

V	Contexto en el Proyecto	Enfoque en la Arquitectura
Volumen	Archivos CSV históricos (ej. facturación mensual) y eventos JSONL intencionalmente fragmentados para simular micro-lotes.	Uso de Parquet particionado , particionado sensato y operaciones de coalesce o repartition para optimizar el rendimiento del motor Spark.
Velocidad	Coexistencia de métricas operativas <i>near real-time</i> y procesos de maestros <i>batch</i> diarios/mensuales.	Implementación del patrón Lambda , combinando <i>Structured Streaming</i> (con ventanas y <i>watermarks</i>) y <i>Jobs Batch</i> independientes.
Variedad	Múltiples formatos originales (CSV, JSONL) y mutación de la estructura de datos (evolución de esquema a partir de ~45 días atrás).	Data Lake con capas definidas: tipificación explícita en Bronze y normalización/compatibilización de esquemas en Silver .
Veracidad	Presencia de datos ruidosos, nulos, tipos ambiguos y valores atípicos (ej. incrementos de costo negativos ocasionales).	Filtros de calidad hacia Quarantine para aislar errores técnicos, y cálculo de <i>flags</i> para derivar excepciones de negocio a la capa Gold (<i>cost_anomaly_mart</i>).
Valor	Necesidad de responder preguntas analíticas específicas de FinOps, Soporte y Producto (GenAI) mediante dashboards.	Capa Gold con <i>Marts</i> de negocio altamente agregados, exportados a Cassandra y listos para responder velozmente mediante sentencias CQL.

4. Flujos de Datos (Data Pipelines)

A continuación se presenta el flujo de datos, dividido lógicamente en las dos ramas de la arquitectura Lambda (Batch y Streaming). Ambas rutas procesan la información a través de las capas del Data Lake (en formato Parquet) y convergen finalmente en la capa de servicio.

4.1. Pipeline Batch (Maestros y Facturación)

Tramo	Herramienta y Función	Notas y Justificación
CSV → Bronze	PySpark (<code>read.csv</code> → <code>cast</code> → <code>write.parquet</code>)	Lectura de maestros y facturación estáticos; particionado por <code>ingest_date</code> o entidad.
Bronze → Silver	PySpark Batch (Filtros y Joins)	Desvío de filas inválidas a Quarantine ; unificación de campos para esquema v1/v2.
Silver → Gold	PySpark Batch (Agregaciones)	Cálculo de métricas mensuales y pre-cálculo de <i>flags</i> de anomalías; escritura en Parquet.
Gold → AstraDB	Spark Cassandra Connector	Carga masiva de los <i>marts</i> finalizados hacia las tablas diseñadas <i>query-first</i> en Cassandra.

4.2. Pipeline Streaming (Eventos de Uso)

Tramo	Herramienta y Función	Notas y Justificación
JSONL → Bronze	Structured Streaming (<code>readStream</code> + esquema explícito + <code>withWatermark</code>)	Ingesta continua de <code>usage_events_stream/*.jsonl</code> ; manejo de <i>late data</i> y dedup por <code>event_id</code> .
Bronze → Silver	Structured Streaming + micro-batches	Desvío a Quarantine ; merges idempotentes para enriquecer eventos en tiempo real.
Silver → Gold	Structured Streaming (Agregaciones por ventana)	Generación de métricas <i>near real-time</i> (ej. consumo GenAI y costos incrementales).
Gold → AstraDB	Spark (<code>foreachBatch</code> + driver Python)	Escritura incremental (<i>upserts</i> o re-escrituras idempotentes) para actualizar dashboards al instante.

5. Supuestos y Riesgos Iniciales

Para delimitar el alcance del diseño preliminar y evitar la sobre-ingeniería, se establecen los siguientes supuestos operativos y se identifican los riesgos técnicos más críticos junto con sus respectivas estrategias de mitigación.

5.1. Supuestos

- **Volumen de datos manejable en desarrollo:** Se asume que, para la fase de construcción y demostración en Google Colab, el volumen total de datos crudos no excederá los límites de memoria/almacenamiento de un único nodo de procesamiento (ej. < 10 GB en total). Para un entorno productivo real, esto escalaría de forma transparente gracias a Spark.
- **Evolución de esquema aditiva:** Se asume que el cambio de esquema (`schema_version` v1 a v2, con la aparición de `carbon_kg` y `genai_tokens`) es estrictamente aditivo y no altera de forma destructiva los tipos de datos de las columnas preexistentes.
- **Conectividad con la Serving Layer:** Se da por sentado que el entorno de ejecución (Colab) tendrá salida a internet estable para establecer la conexión mediante el driver de Python/conector de Spark hacia los servidores de AstraDB (Cassandra).

5.2. Riesgos y Mitigaciones

- **Riesgo 1: Cuellos de botella de memoria (OOM) en cruces de datos (Joins).** Al unir la tabla de eventos de uso (alta cardinalidad) con maestros como organizaciones o usuarios en la capa Silver, Spark podría quedarse sin memoria.
 - *Mitigación:* Forzar el uso de *Broadcast Joins* para las tablas de dimensiones pequeñas (maestros) y asegurar un particionado adecuado (por fecha o servicio) al leer desde Bronze.
- **Riesgo 2: Latencia en el procesamiento de Streaming sin un broker dedicado.** Al no contar con herramientas como Kafka o Pub/Sub y depender de leer archivos `.jsonl` de un directorio para simular el stream, se puede generar latencia o bloqueos en la lectura.
 - *Mitigación:* Ajustar el *trigger* del micro-batch a intervalos controlados (ej. 1 a 5 minutos) y confiar en el manejo robusto de *watermarks* para acotar el tiempo que Spark espera por datos tardíos (*late data*).
- **Riesgo 3: Duplicación de datos ante fallos del pipeline.** Si una celda de Colab falla a la mitad de una escritura hacia Gold o AstraDB, al re-ejecutar se podrían duplicar registros de costos o uso.
 - *Mitigación:* Implementar estrictas medidas de **idempotencia** exigidas por el negocio: configurar directorios de *checkpointing* persistentes, usar `event_id` como clave natural y ejecutar *upserts* en lugar de simples *appends* al cargar en Cassandra.

6. Estimación de esfuerzo y recursos

Frente	Alcance incluido	Horas estimadas
Arquitectura + notebooks Spark	Lake por zonas, batch + Structured Streaming, quarantine, reproducibilidad en Colab	25–45
Gold + calidad de datos	Marts FinOps/Soporte/GenAI, reglas DQ, anomalías (percentiles robustos), idempotencia	15–30
Cassandra / AstraDB	Diseño tablas por consulta, carga desde Spark, 5 consultas demo con capturas	12–22
Demo + comunicación	Presentación, video, ajustes de credenciales y paths	10–18
Buffer integración	Colab + I/O + re-ejecuciones por esquema/credenciales	10–20
Total orientativo		≈ 70–135 h equipo

Roles sugeridos para el equipo:

- **Data Engineer (Ingesta y Pipelines):** Encargado de la lectura de fuentes, el particionado, y la implementación técnica de las ramas batch (maestros y facturación) y *Structured Streaming* (eventos de uso) hacia Bronze.
- **Data Engineer (Calidad y Transformación):** Responsable de la lógica de negocio en la capa Silver, la gestión de la evolución de esquema (v1 a v2), las reglas hacia *Quarantine* y el cálculo estadístico de anomalías.
- **Data Architect / Integración End-to-End:** Encargado del modelado NoSQL *query-first* en Cassandra, la estrategia de carga (upserts/idempotencia), la ejecución de las consultas CQL de prueba y la orquestación general del notebook para asegurar que el pipeline funcione como un producto cohesivo.

7. Conclusión

El diseño preliminar detallado en este documento establece una base sólida, visual y accionable para la implementación del proyecto *Cloud Provider Analytics*. La elección de una **Arquitectura Lambda** resulta ser el enfoque más pragmático para satisfacer el doble requerimiento de latencia operativa (*near real-time*) y procesamiento histórico pesado, evitando introducir complejidades innecesarias en esta fase temprana de validación.

Al estructurar el almacenamiento intermedio en Parquet bajo el modelo de zonas (Bronze, Silver, Gold) y delegar la capa de servicio a una base de datos de alta velocidad como AstraDB, se garantiza la separación de responsabilidades. El diseño propuesto asegura la resiliencia del pipeline ante datos inconsistentes, soporta la evolución dinámica de los esquemas, e implementa la idempotencia necesaria para reprocesar información sin comprometer la integridad.

Con los flujos de datos delineados y los riesgos iniciales mitigados, el equipo cuenta con una hoja de ruta clara para iniciar el desarrollo del código en PySpark, con la certeza de que la infraestructura resultante será capaz de alimentar de manera confiable los tableros analíticos de FinOps, Soporte y Producto.